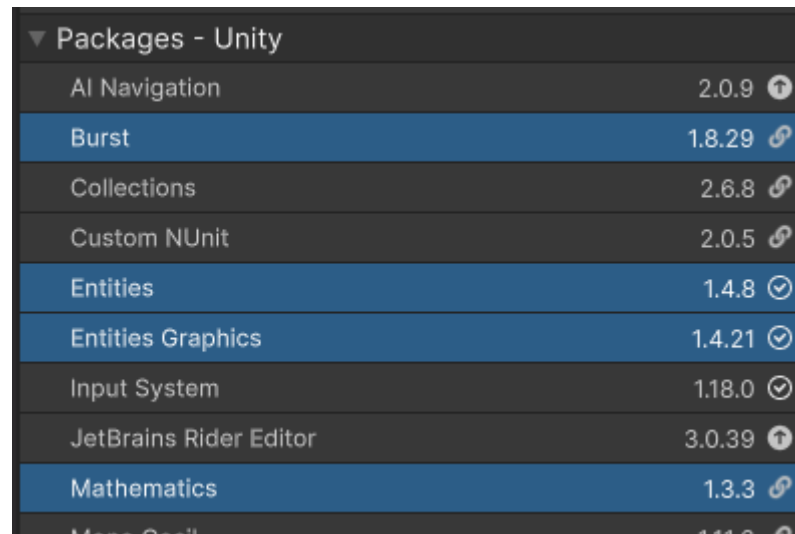
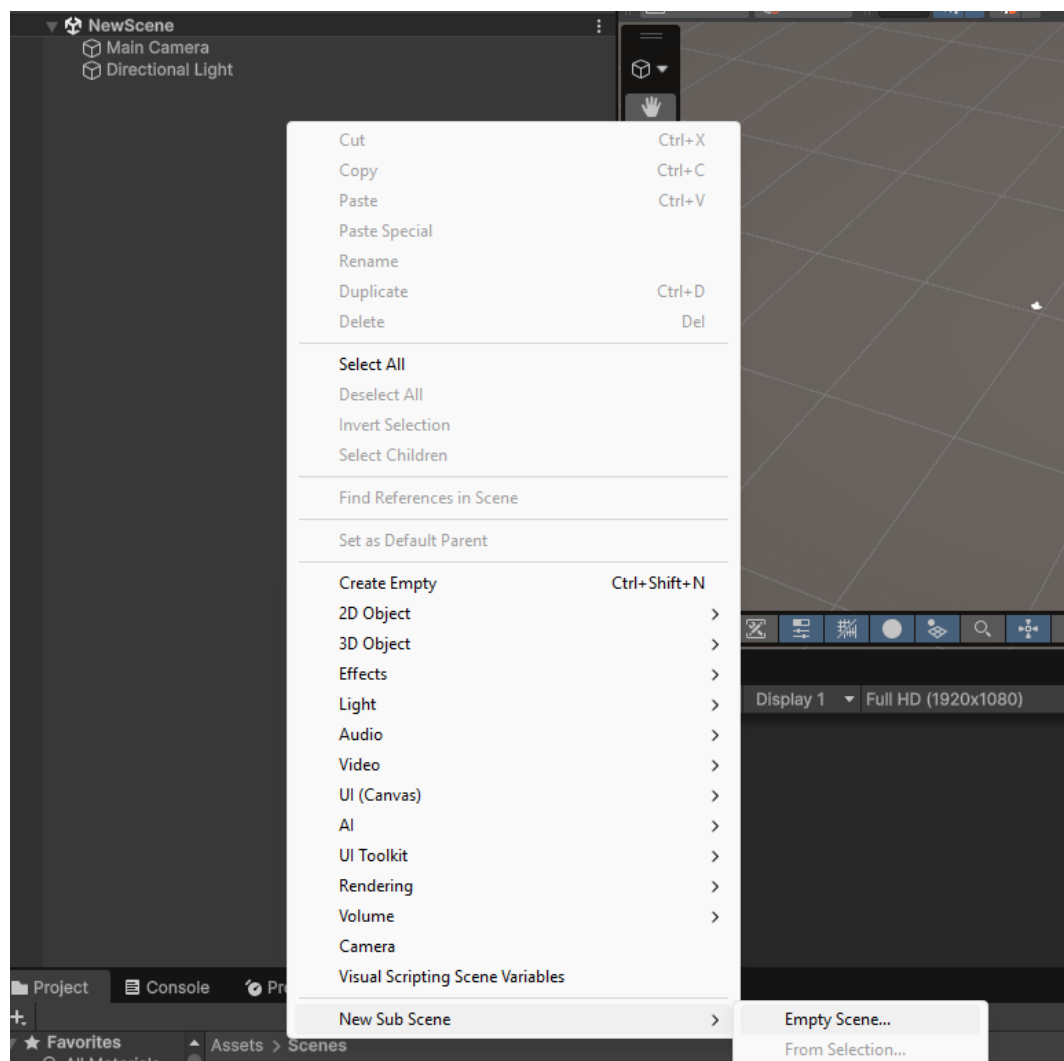


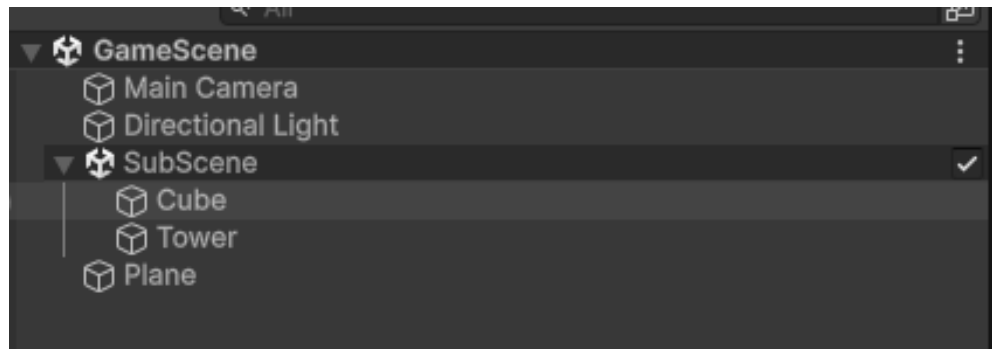
Перед началом работы с Unity DOTS убедитесь, что у вас установлены пакеты Entities, Entities Graphics, Mathematics и Burst.



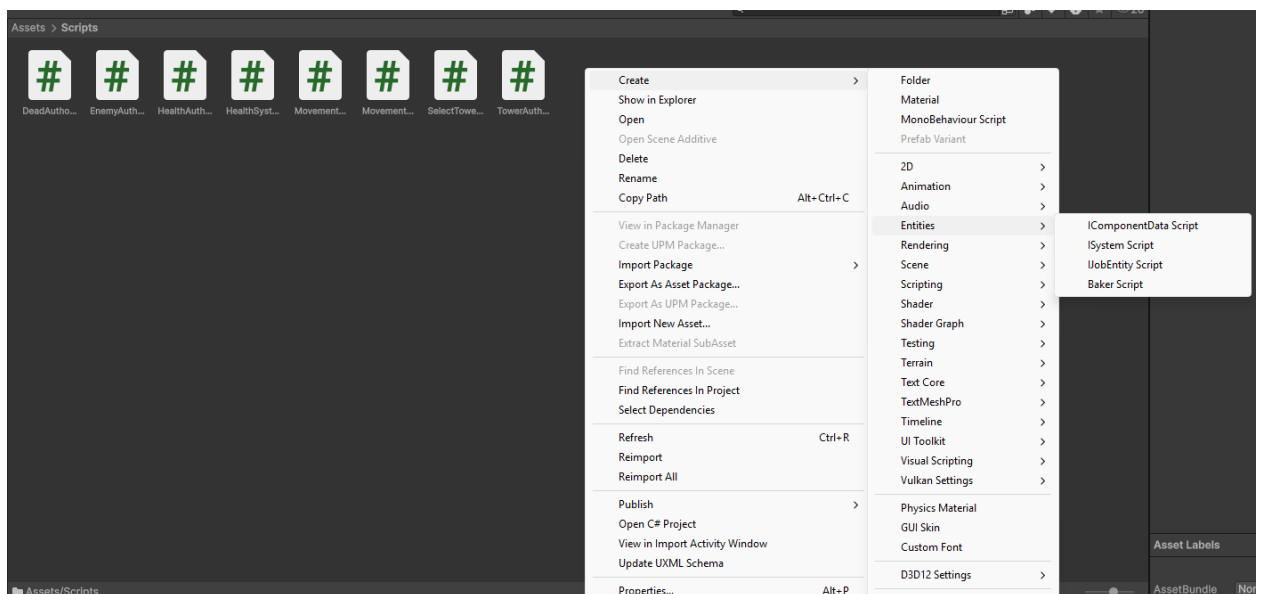
Entity живут в своей отдельной сцене, так что на своей сцене следует создать сцену, где и будут происходить все операции с entity.



Если какой-то GameObject будет запекаться в entity, он должен быть в этой сцене.



Для запекания компонентов, а также систем существуют готовые заготовки скриптов, как, например, существует заготовка для MonoBehaviour со Start и Update. Здесь аналогично.



Компонент это struct, реализующий интерфейс IComponentData. Если нужно, чтобы компонент был ещё и выключаемым, то помимо IComponentData на этой структуре должен быть IEnableableComponent.

```
Ссылка: 0
public struct MyComponent1 : IComponentData
{
    ...
}

Ссылка: 0
public struct MyComponent2 : IComponentData, IEnableableComponent
{
    ...
}
```

Теперь к тому, как этот компонент запечь. В созданном шаблоне Baker скрипта нужно для начала объявить entity, в который превратится GameObject в рантайме. TransformUsageFlags.Dynamic как параметр метода GetEntity - наиболее часто используемый вариант, который подходит для большинства случаев. Но если вы собираетесь непропорционально менять размеры entity, то вместо него подойдёт NonUniformScale, если же entity вообще не важно находиться где-то в координатной сетке и иметь размер, то подойдёт None.

```
class MyComponentAuthoring : MonoBehaviour
{
}

Ссылка: 0
class MyComponentAuthoringBaker : Baker<MyComponentAuthoring>
{
    Ссылка: 0
    public override void Bake(MyComponentAuthoring authoring)
    {
        Entity entity = GetEntity(TransformUsageFlags.Dynamic);
    }
}
```

Теперь методом AddComponent запекаем на этого entity новый экземпляр нужного компонента и, если необходимо, передаём в него параметры.

```
class MyComponentAuthoring : MonoBehaviour
{
    public int _testValue;
}

Ссылка: 0
class MyComponentAuthoringBaker : Baker<MyComponentAuthoring>
{
    Ссылка: 0
    public override void Bake(MyComponentAuthoring authoring)
    {
        Entity entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new MyComponent1()
        {
            TestValue = authoring._testValue
        });
    }
}

Ссылка: 1
public struct MyComponent1 : IComponentData
{
    public int TestValue;
}
```

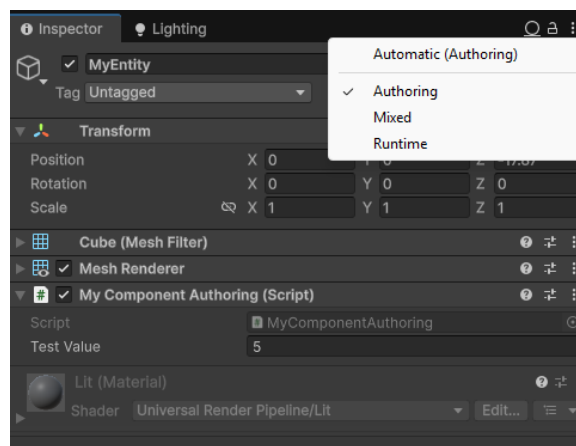
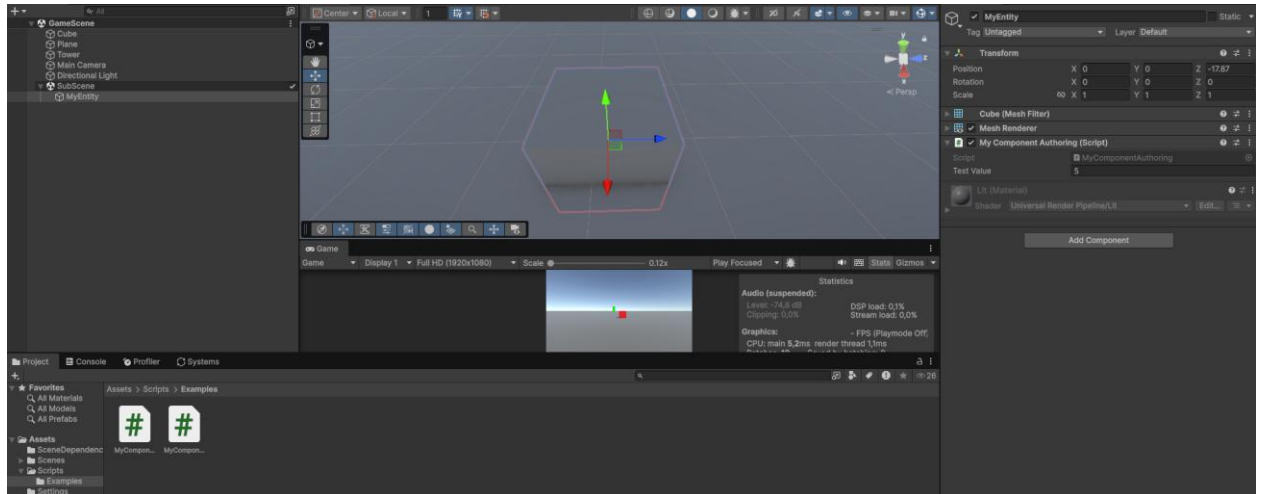
Чтобы задать для выключаемого компонента стартовое состояние - можно использовать метод `SetComponentEnabled`.

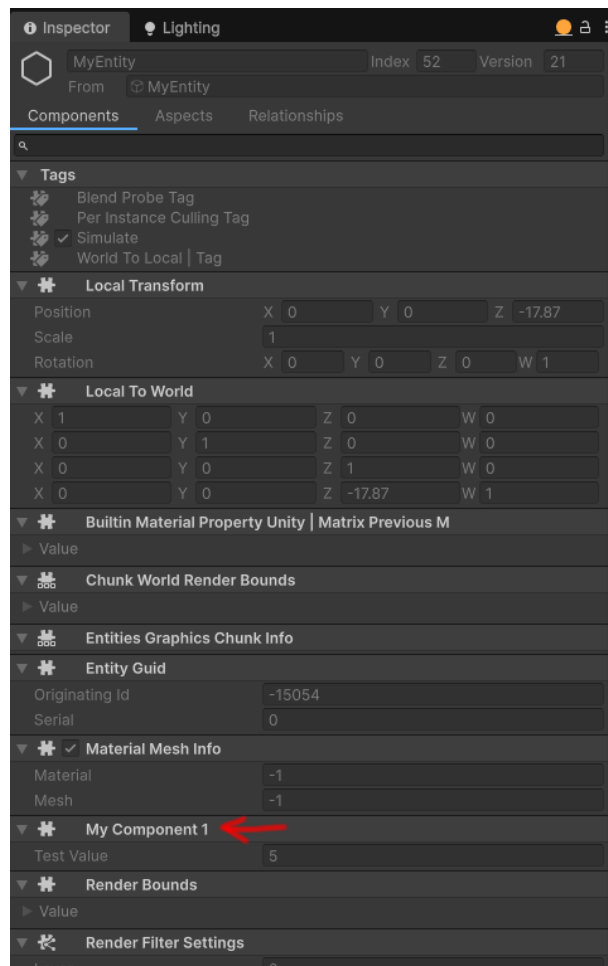
```
class MyComponentAuthoring : MonoBehaviour
{
}

// Ссылка: 0
class MyComponentAuthoringBaker : Baker<MyComponentAuthoring>
{
    // Ссылка: 0
    public override void Bake(MyComponentAuthoring authoring)
    {
        Entity entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new MyComponent2());
        SetComponentEnabled<MyComponent2>(entity, false);
    }
}

// Ссылка: 2
public struct MyComponent2 : IComponentData, IEnableableComponent
{
}
```

Теперь если разместить Baker скрипт на GameObject, то в runtime, когда он станет entity, вы найдёте на нём нужный компонент (переключение на Runtime режим просмотра данных находится в правом верхнем углу инспектора).





Также важно, что в DOTS используется библиотека `Unity.Mathematics`, а значит, например, вместо `Vector2` и `Vector3` следует использовать `float2` и `float3`. Вместо знакомых вам математических библиотек типа `Mathf` или просто `Math` с большой буквы следует использовать `math` с маленькой буквы. Внутри класса `math` вы, как и раньше сможете найти необходимые математические методы.

```

Ссылка: 1
public struct MyComponent1 : IComponentData
{
    public int TestValue;

    //public Vector3 Position; // не так!!!
    public float3 Position; // вот так
}

```

```

[BurstCompile]
Ссылка: 0
public void OnUpdate(ref SystemState state)
{
    foreach(var (comp1, comp2) in SystemAPI.Query<RefRW<MyComponent1>, RefRO<MyComponent2>>())
    {
        float3 coolVector = new float3(10f, 18f, -2f);
        float3 normalizedVector = math.normalizesafe(coolVector);
    }
}

```

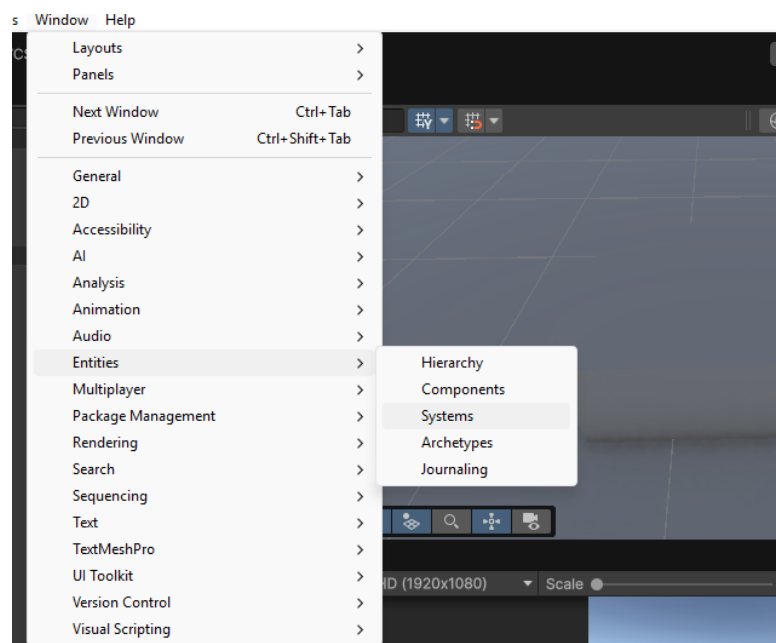
Также, учитывая, что OnUpdate в системах по умолчанию отработывает раз в кадр, вам может понадобиться Time.deltaTime. Вместо привычного вам Time.deltaTime, пожалуйста, используйте SystemAPI.Time.DeltaTime.

```
[BurstCompile]
Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach(var (transform, comp1, comp2) in SystemAPI.Query< RefRW<LocalTransform>, RefRO<MyComponent1>, RefRO<MyComponent2>>>())
    {
        float3 coolVector = new float3(10f,18f,-2f);

        float3 normalizedVector = math.normalizesafe(coolVector);

        transform.ValueRW.Position += new float3(0,0,comp1.ValueRO.TestValue * SystemAPI.Time.DeltaTime);
    }
}
```

Окно для просмотра систем находится в Window > Entities > Systems. В нём можно узнать порядок выполнения систем, количество entity, которое сейчас обрабатывает каждая система, а также то, сколько времени (производительности) съедает каждая система.

A screenshot of the Unity Systems window. It displays a list of systems and their performance metrics. The table has columns for World, Namespace, Entity Count, and Time (ms).

	World	Namespace	Entity Count	Time (ms)
Initialization				
Update				
Simulation System Group	Default World	Unity.Entities		0,00
Begin Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0,00
Fixed Step Simulation System Group	Default World	Unity.Entities		0,00
Variable Rate Simulation System Group	Default World	Unity.Entities		0,00
Select Tower System	Default World	Unity.Entities		0,00
My Example System	Default World	Unity.Entities		0,00
Companion Game Object Update System	Default World	Unity.Entities		0,00
Transform System Group	Default World	Unity.Transforms		0,00
Movement System	Default World	Unity.Entities		0,00
Health System	Default World	Unity.Entities		0,00
Companion Game Object Update Transform System	Default World	Unity.Entities		0,00
Companion Game Object Live Baking Init System	Default World	Unity.Entities		0,00
Late Simulation System Group	Default World	Unity.Entities		0,00
End Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0,00
Pre Late Update				

Чтобы перебрать в системе entity с определёнными компонентами, используется обычный цикл foreach, где нужные компоненты задаются через

SystemAPI.Query. Сами компоненты в этом Query записываются как Generic параметры структур RefRO или RefRW - в зависимости от того, будем ли мы только читать данные из компонента или ещё и менять их.

```
[BurstCompile]
//Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach(var (transform, comp1, comp2) in SystemAPI.Query< RefRW<LocalTransform>, RefRO<MyComponent1>, RefRO<MyComponent2>>())
    {
        float3 coolVector = new float3(10f,18f,-2f);

        float3 normalizedVector = math.normalizesafe(coolVector);

        transform.ValueRW.Position += new float3(0,0,comp1.ValueRO.TestValue * SystemAPI.Time.DeltaTime);
    }
}
```

Если нужен доступ к самому entity, то к SystemAPI.Query нужно добавить метод WithEntityAccess, и добавить переменную цикла для entity.

```
[BurstCompile]
//Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach(var (transform, comp1, comp2, entity) in SystemAPI.Query< RefRW<LocalTransform>, RefRO<MyComponent1>, RefRO<MyComponent2>>().WithEntityAccess())
    {
        float3 coolVector = new float3(10f,18f,-2f);

        float3 normalizedVector = math.normalizesafe(coolVector);

        transform.ValueRW.Position += new float3(0,0,comp1.ValueRO.TestValue * SystemAPI.Time.DeltaTime);
    }
}
```

Если логика сложная, и выполнять её «в холостую» будет стоить производительности, то всегда можно добавить проверку через if, но также существует такая опция, как метод WithChangeFilter – он подразумевает, что логика отработает только для тех entity, у которых поменялись данные внутри конкретного компонента. Но стоит учитывать, что он не гарантирует, что поменялись данные из компонента конкретного entity. Возможно, поменялись данные другого entity из этого же чанка памяти.

```
[BurstCompile]
//Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach(var (transform, comp1, comp2, entity) in SystemAPI.Query< RefRW<LocalTransform>, RefRO<MyComponent1>, RefRO<MyComponent2>>().WithEntityAccess().WithChangeFilter<MyComponent1>())
    {
        float3 coolVector = new float3(10f,18f,-2f);

        float3 normalizedVector = math.normalizesafe(coolVector);

        transform.ValueRW.Position += new float3(0,0,comp1.ValueRO.TestValue * SystemAPI.Time.DeltaTime);
    }
}
```

Альтернативой привычному вам методу gameObject.GetComponent является специальная структура ComponentLookup. Если нужно, имея конкретного entity, получить его конкретный компонент, то создаётся ComponentLookup, в Generic параметр которого нужно передать нужный

компонент. В параметр метода `SystemAPI.GetComponentLookup` можно передать булево значение, которое будет определять, нужен ли компонент, найденный с помощью `ComponentLookup` только для чтения или для записи тоже. Функционал методов `ComponentLookup` вполне интуитивный исходя из их названий.

```
[BurstCompile]
Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach (var (transform, comp1, comp2, entity) in SystemAPI.Query<RefRW<LocalTransform>, RefRO<MyComponent1>, RefRO<MyComponent2>>().WithEntityAccess().WithChangeFilter<MyComponent1>())
    {
        float3 coolVector = new float3(10f, 18f, -2f);

        float3 normalizedVector = math.normalizesafe(coolVector);

        transform.ValueRW.Position += new float3(0, 0, comp1.ValueRO.TestValue * SystemAPI.Time.DeltaTime);

        ComponentLookup<Movement> movementLookup = SystemAPI.GetComponentLookup<Movement>(false);

        Movement entityMovement = movementLookup[entity]; // если уверены, что такой компонент есть на этом entity
        bool result = movementLookup.HasComponent(entity); // проверить, есть ли такой компонент на этом entity
        bool result2 = movementLookup.TryGetComponent(entity, out Movement outMovement, out bool entityExists); // TryGetComponent, но для entity (вроде, интуитивно)
    }
}
```

Как и в Baker скрипте для изменения состояния выключаемого компонента в системе используется метод `SetComponentEnabled`. В системе этот метод можно найти в `state.EntityManager`.

```
[BurstCompile]
Ссылка 0
public void OnUpdate(ref SystemState state)
{
    foreach (var (health, entity) in SystemAPI.Query<RefRO<Health>>().WithDisabled<Dead>().WithEntityAccess().WithChangeFilter<Health>())
    {
        if (health.ValueRO.CurrentHealth <= 0)
        {
            state.EntityManager.SetComponentEnabled<Dead>(entity, true);
        }
    }
}
```

Иногда чёткий порядок выполнения систем в каждом отдельном кадре критически важен. По умолчанию же системы обрабатывают в случайном порядке (который можно посмотреть в ранее упомянутом окне просмотра систем). Чтобы чётко задать, что одна система обрабатывает после или перед другой, нужны атрибуты `UpdateBefore` и `UpdateAfter`, которые записываются перед системой.

```
using Unity.Burst;
using Unity.Entities;

[UpdateAfter(typeof(MovementSystem))]
Ссылка 1
partial struct HealthSystem : ISystem
{
    [BurstCompile]
    Ссылка 0
    public void OnUpdate(ref SystemState state)
    {
        foreach (var (health, entity) in SystemAPI.Query<RefRO<Health>>().WithDisabled<Dead>().WithEntityAccess().WithChangeFilter<Health>())
        {
            if (health.ValueRO.CurrentHealth <= 0)
            {
                state.EntityManager.SetComponentEnabled<Dead>(entity, true);
            }
        }
    }
}
```

Если вы уверены, что Entity с некоторым компонентом точно всего 1, то с целью его нахождения можно использовать метод `GetSingleton` или `GetSingletonEntity` – в зависимости от потребностей (`TryGetSingleton` и `TryGetSingletonEntity` тоже существуют).

```
[BurstCompile]
Ссылка 0
public void OnUpdate(ref SystemState state)
{
    if(!SystemAPI.TryGetSingletonEntity<Tower>(out Entity towerEntity))
    {
        return;
    }

    ComponentLookup<LocalTransform> transformLookup = SystemAPI.GetComponentLookup<LocalTransform>(true);
    if(transformLookup.TryGetComponent(towerEntity, out LocalTransform towerTransform, out bool entityExists) && entityExists)
    {
        foreach (var movement in SystemAPI.Query<RefRW<Movement>>())
        {
            movement.ValueRW.TargetPosition = towerTransform.Position;
        }
    }
}
```